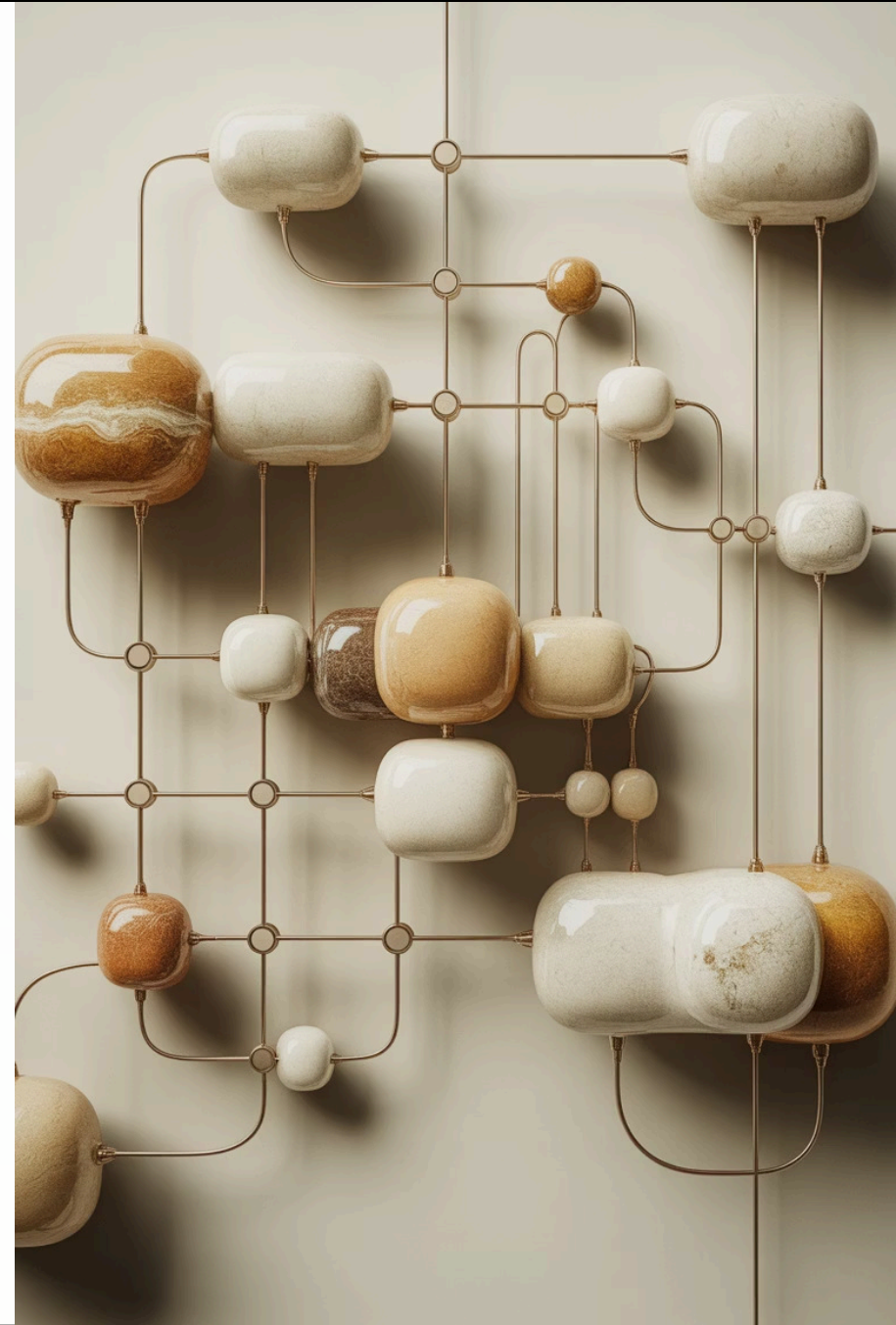
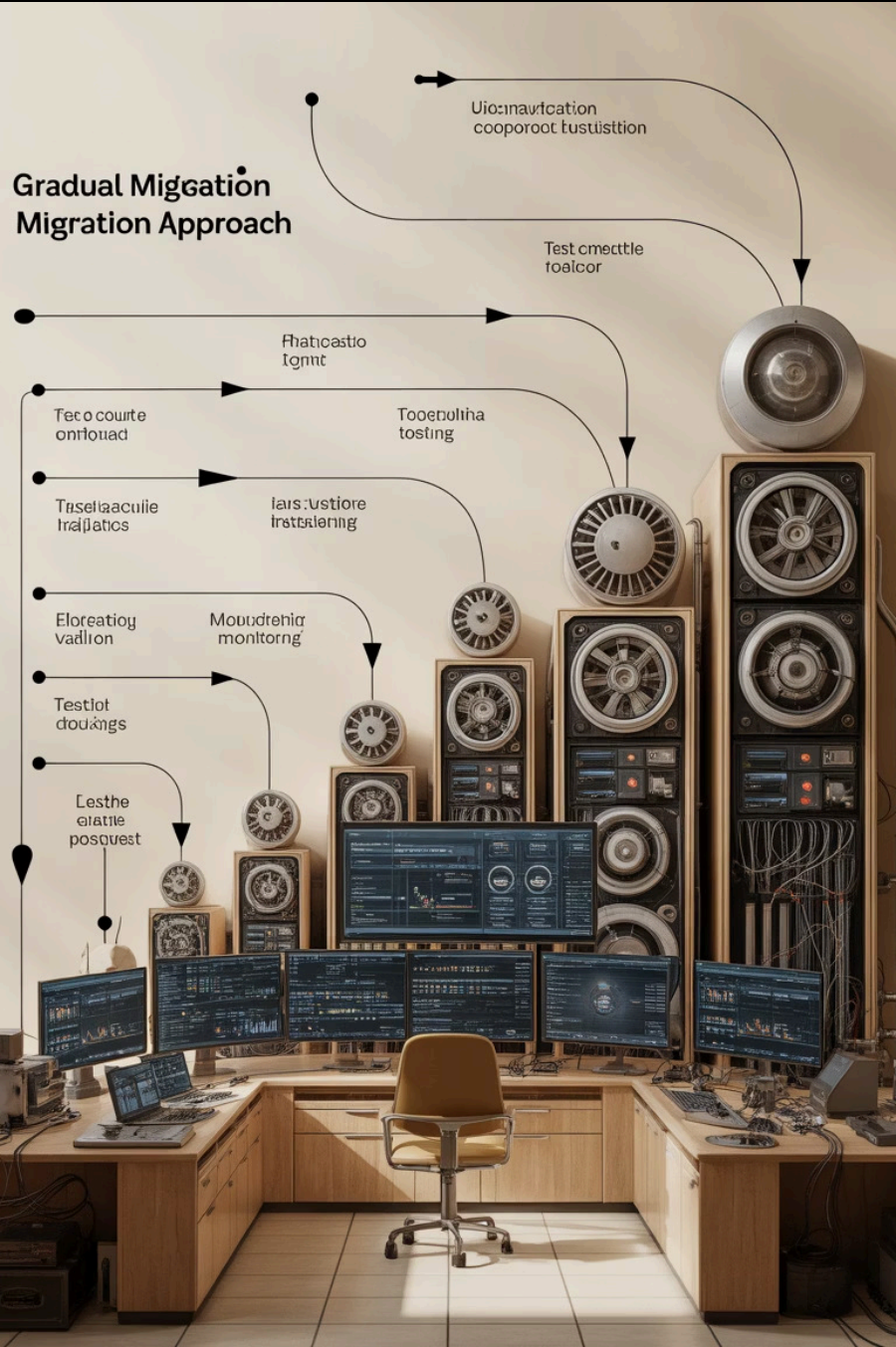


# Objetivo da Migração e Roteiro Sugerido

Este material de referência detalha a justificativa e a sequência de passos proposta para a migração gradual de microserviços. O objetivo é fornecer um guia prático, com comandos e checklists, para cada etapa do processo de refatoração, visando uma transição controlada e bem-sucedida.

O roteiro estratégico apresentado explica por que cada fase é crucial para minimizar riscos e maximizar a eficiência da migração, estabelecendo uma base sólida para o futuro da arquitetura.





# Visão Estratégica da Migração

## Abordagem Gradual

A migração segue uma estratégia incremental, começando com componentes simples e evoluindo para sistemas mais complexos. Esta abordagem reduz riscos e permite aprendizado contínuo.

## Validação Constante

Cada fase inclui validação rigorosa através de testes e monitoramento, garantindo que a transição não comprometa a estabilidade dos sistemas existentes.

# Provedor Mínimo (PoC)



## Justificativa

Iniciar com um alvo simples e controlado permite implementar rapidamente um contrato mínimo, validando o conceito de "service" sem a complexidade total do framework do monólito.



## Resultado Esperado

Um servidor pequeno utilizando Express ou Nest.js em sua forma mais básica, que responde aos endpoints necessários com respostas mockadas, comprovando a comunicação e o contrato.

Esta abordagem demonstra a viabilidade da estratégia e estabelece uma base sólida para as próximas fases da migração, minimizando riscos e complexidade inicial.







# Instruções para o Provedor Mínimo

## Configuração do Serviço de Usuários

Crie uma PoC de "provider minimal" para o serviço de users (Express) dentro do monorepo em users-service. Esta implementação servirá como prova de conceito para validar a arquitetura proposta.

01

### Ambiente

Node 18+ com preferência por TypeScript. Se o pacote já usa TS, mantenha a linguagem.

02

### Localização

Implementar em users-service, sem modificar o monolito users existente.

03

### Contrato

Usar openapi.yaml como referência, versionando com openapi: 3.1.0.

# Endpoints a Implementar

1

GET /health

**Retorno:** 200 OK

**Body:** { "status": "ok" }

Endpoint simples para verificar a saúde do serviço.

2

POST /users

**Body:** { email: string, name?: string, password?: string }

**Dados válidos:** 201 { id, email, name }

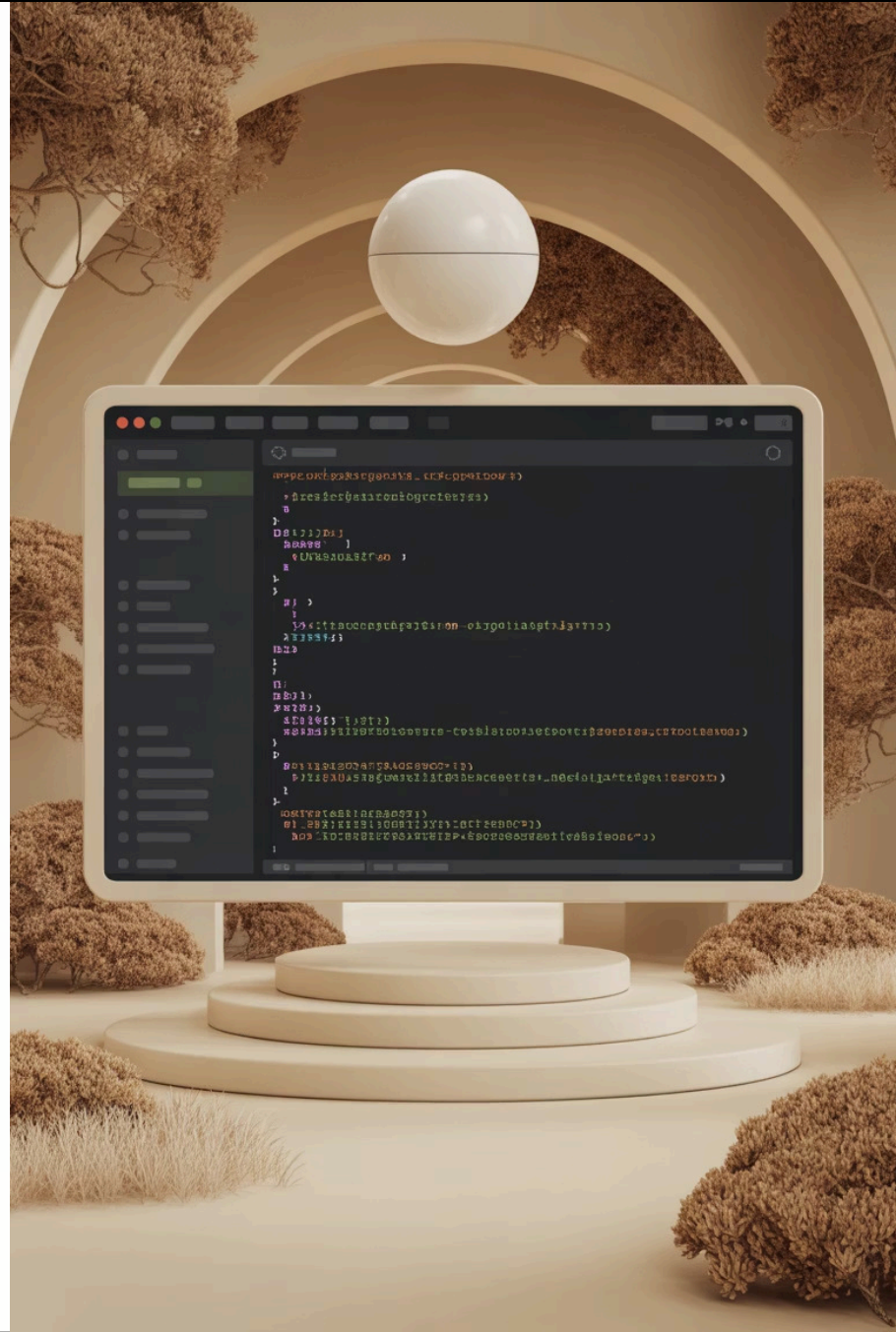
**Dados inválidos:** 400 { message: "Invalid payload" }

3

GET /users/:id

**Usuário encontrado:** 200 { id, email, name }

**Não encontrado:** 404 { message: "User not found" }



Centb Erpo.

Configuration of a  
Minimal User Service  
within Monepo

BOPE NOEUMIL SIMYONEN  
BOWMER

CONEOREPO

COIRAC  
A11mdeit Aclerats  
Ribimel Services

CONTRACT REQUIREMENTS  
Pawceasonolecteritesic  
perctetnom teurnollie neednd  
Rat'eostbou

CONTACT REQUIREENTS  
Reet'esstom eltrinesd TLH  
Boet'ot'ne:cafrone  
Racour stat

ARRESIA  
Móthai 1K Data

PRISIQUEOIN  
Dmcoahne omelectreplece  
Pie a.cace coetie f'fectis

PENACREDO.

LOEGCIRIK  
Cotie Tórie Sísicoeek botisak

REMLIREOIN  
Acéltie jio ttoirteat'inol  
Rentit coesú

CONTRACT  
Debt'ant'et'ion'ora  
Reet'esstom'olecabe  
Ret'ier b'alen

ABHDEA  
LOGER OOEBLAPENINS

# Fluxo de Verificação de Saúde

## Verificação de Status

Retorna o status atual do serviço, confirmando que está operacional e pronto para receber requisições.

## Tempo de Atividade

Monitora o uptime do sistema, fornecendo métricas sobre a estabilidade e disponibilidade do serviço.

## Checagem de Dependências

Verifica a conectividade com banco de dados, cache e filas, garantindo integridade do ecossistema.



# Especificações de Implementação



## Servidor Express Básico

Configurar servidor com middlewares `express.json()` para parsing de JSON e `cors()` para gerenciamento de políticas de segurança entre origens.



## Respostas Mockadas

Implementar respostas totalmente mockadas, sem persistência. IDs podem ser strings estáticas ("mocked-user-id") ou gerados com `uuid/v4`.



## Exportação da App

Exportar a instância Express (`module.exports` ou `export default app`) para permitir testes com `supertest`.



## Ponto de Entrada

Criar `src/server.ts` ou `src/index.ts` que chama `app.listen()` para executar via `npm run dev`.

## MINTEUMNTIM PROVIDER SERVICE

WHOLESALE MANUFACTURING AND DISTRIBUTION OF HIGH-QUALITY, AFFORDABLE, AND SUSTAINABLE PRODUCTS AND SERVICES.

| IMPLEMENTING FOO LOWIBCEVTIR | ONTOPCEPRUBURG       |
|------------------------------|----------------------|
| <b>AWEMENCTALEVCURRENT</b>   | <b>POTURANOSHING</b> |
| <b>ERSIAM</b>                | <b>LOABYS</b>        |
| <b>CONTRACT DOANG</b>        | <b>VIIIG</b>         |
| <b>IVIS</b>                  | <b>DENUTACTIIOU</b>  |
| <b>BBTHJIG.</b>              | <b>JOUAL</b>         |



# OpenAPI: O Contrato de Serviço

O arquivo `packages/users-service/openapi.yaml` é um documento crucial que serve como o contrato formal para o serviço de usuários. Baseado na especificação OpenAPI 3.1.0, ele descreve de forma clara e legível todas as capacidades e a interface de comunicação do serviço.

Sua existência é fundamental para garantir a **interoperabilidade**, facilitar o desenvolvimento colaborativo e automatizar processos importantes, como a geração de documentação e clientes de API. Ele é o ponto de verdade sobre como outros sistemas podem interagir com o `users-service`.



# Estrutura do OpenAPI

## Info

Contém metadados básicos sobre a API, como título (Users Service PoC) e versão (0.1.0).

## Paths

Descreve os caminhos (endpoints) da API e as operações HTTP associadas (GET, POST, etc.).

## Components/Schemas

Define os modelos de dados (DTOs) utilizados nas requisições e respostas, como CreateUser e User.



## Specificațion iote eeteicnajor

[illegible]

Robust to verbs, too, organic extraneous conditions.

[illegible]

Assaggio.

[illegible]

THE ORIGINATOR

784. ACINE STIJC

ENCLOSURE

### SUGARY DRINK

## CreateUser

User

Representa um usuário registrado no sistema:

- **email** (obrigatório): Endereço de e-mail do usuário
- **name** (opcional): Nome completo do usuário
- **password** (opcional para PoC): Senha de acesso
- **id**: Identificador único do usuário
- **email**: Endereço de e-mail cadastrado
- **name**: Nome do usuário no sistema



# Benefícios da Especificação OpenAPI



## Guia para Implementação

A especificação serve como um blueprint para o desenvolvimento dos controladores e DTOs, garantindo que a implementação esteja alinhada com o contrato definido.



## Base para Testes de Contrato

Permite criar testes de contrato automatizados que verificam se a implementação da API realmente atende ao prometido, prevenindo quebras em integrações.



## Documentação Versionada

Mantém a documentação da API versionada junto com o código, garantindo alinhamento entre especificações e versões do serviço.



## Geração de Clientes

Facilita a geração automática de SDKs e stubs de cliente em diversas linguagens, acelerando o desenvolvimento de sistemas consumidores.

# O que é OpenAPI?

OpenAPI é um padrão poderoso e independente de framework para descrever APIs RESTful de forma padronizada. Ele fornece uma linguagem comum para que tanto humanos quanto máquinas possam entender as capacidades de uma API sem a necessidade de acesso ao código-fonte ou documentação adicional.

- Define os endpoints (paths) disponíveis e os métodos HTTP associados
- Detalha os parâmetros que podem ser enviados (query, path, header, body), seus tipos e validações
- Especifica os formatos esperados para requisições e respostas e os esquemas dos dados
- Não está vinculado a uma linguagem de programação ou framework específico



# Abordagens de Integração OpenAPI

## Code-First

O código da API é a fonte primária de verdade. A especificação OpenAPI é gerada automaticamente a partir de anotações no código-fonte (ex: `@nestjs/swagger`). Isso garante que a documentação esteja sempre sincronizada com a implementação, mas o design da API é ditado pelo código.

## Contract-First

O documento `openapi.yaml` é a fonte da verdade e é criado antes de qualquer implementação. A partir dele, são gerados clientes, stubs de servidor e modelos de dados. Essa abordagem promove um design de API robusto e colaborativo, onde o contrato é definido e acordado antes do desenvolvimento.

No nosso repositório, observamos uma abordagem híbrida: a aplicação principal utiliza `@nestjs/swagger` para gerar documentação em tempo de execução, enquanto cada pacote de microserviço mantém um arquivo `openapi.yaml` estático.

# Por que Manter um openapi.yaml Estático?

## Testes de Contrato Robustos

A existência do openapi.yaml permite a criação de testes de contrato automatizados que verificam se a implementação adere estritamente ao contrato, prevenindo regressões.

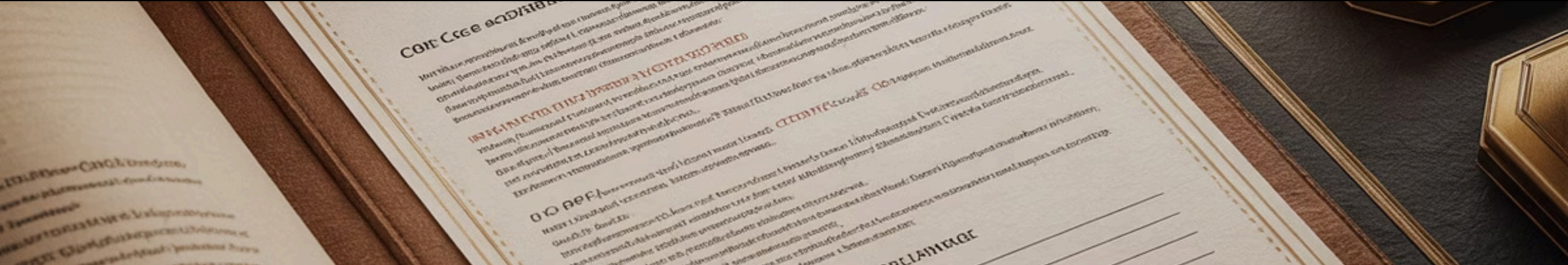
## Geração Simplificada

Um contrato estático é ideal para a geração automática de SDKs em diversas linguagens, acelerando o desenvolvimento e minimizando erros de integração.

## Material Didático e PoC

Os arquivos atuam como guia estático e autoexplicativo, servindo como material didático para novos desenvolvedores ou para propósitos de Prova de Conceito.



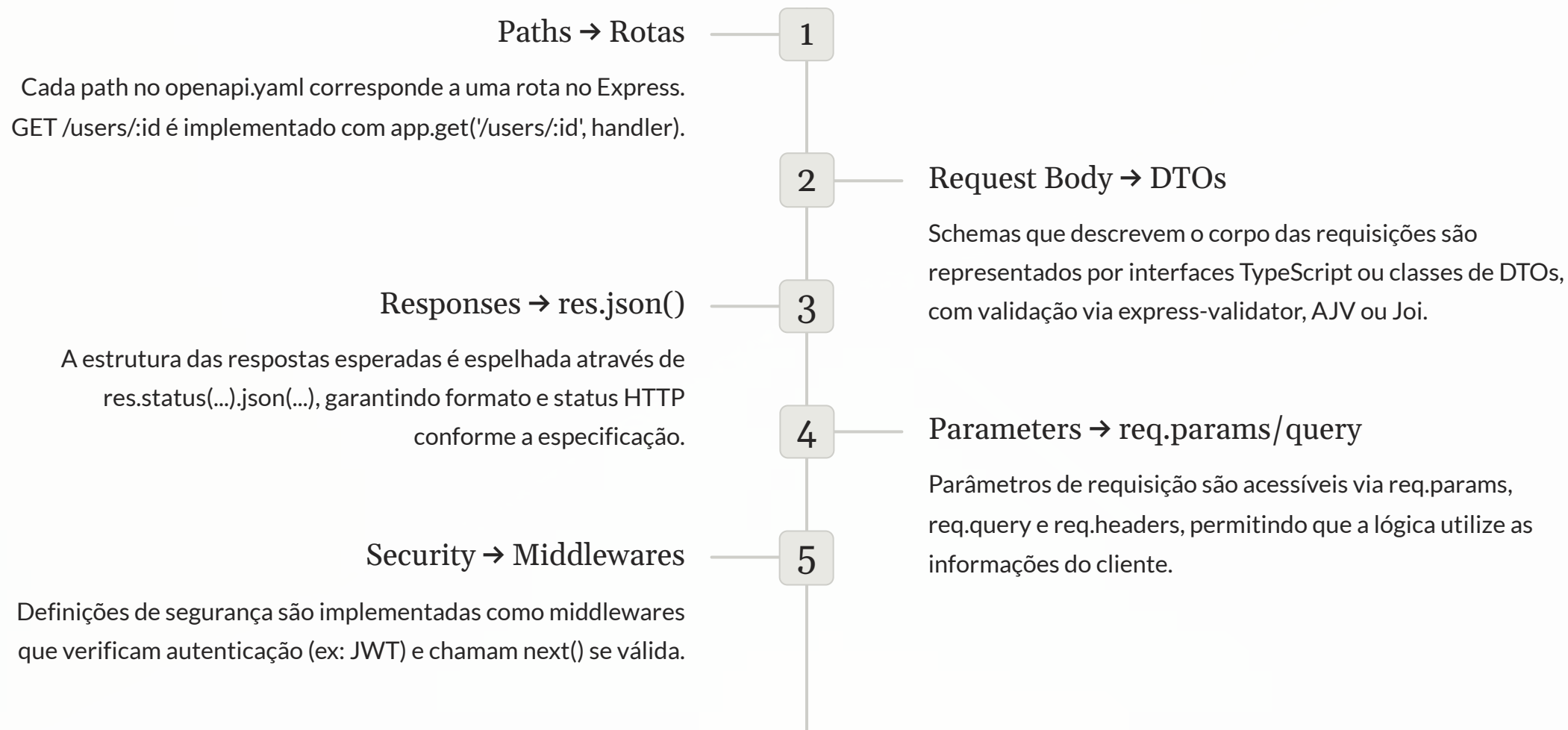


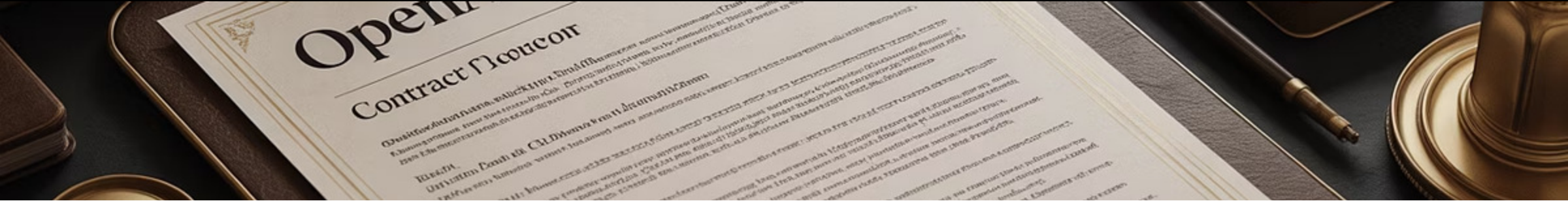
# Mapeando OpenAPI para Express

Para desenvolvedores que buscam integrar a especificação OpenAPI em suas aplicações Express.js, é fundamental compreender como cada elemento do contrato YAML se traduz em componentes de código. Esta seção detalha essa correspondência, oferecendo um guia claro para a implementação.



# Correspondências OpenAPI-Express





## Exemplo Prático: Rotas e Validação

Este exemplo demonstra a criação de endpoints RESTful com Express, manipulação de parâmetros e validação usando express-validator:

```
const express = require('express');
const { body, param, validationResult } = require('express-validator');

const router = express.Router();

router.get('/:id',
  param('id').isString().notEmpty(),
  (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(400).json({ errors: errors.array() });
    const { id } = req.params;
    res.json({ id, email: 'user@example.com', name: 'User' });
  }
);
```

## AUTOMATION VALIDATION

# Validação Automática com express-openapi-validator

Como alternativa à validação manual, é possível automatizar o processo integrando o serviço Express diretamente com o contrato openapi.yaml. A biblioteca express-openapi-validator oferece uma solução poderosa para validar requisições e respostas automaticamente em runtime.

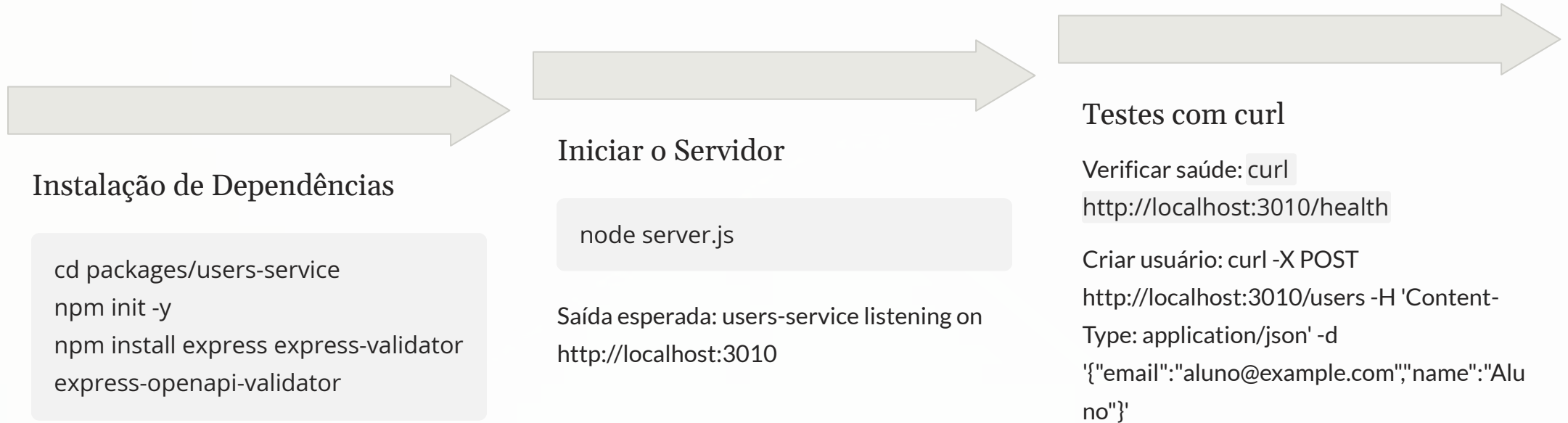
```
const OpenApiValidator = require('express-openapi-validator');

app.use(
  OpenApiValidator.middleware({
    apiSpec: path.join(__dirname, 'packages/users-service/openapi.yaml'),
    validateRequests: true,
    validateResponses: false,
  })
);
```

Esta abordagem reduz código boilerplate e centraliza a validação na especificação OpenAPI, facilitando manutenção e garantindo consistência entre documentação e implementação.



# Execução e Testes do Serviço



## Instalação de Dependências

```
cd packages/users-service  
npm init -y  
npm install express express-validator  
express-openapi-validator
```

## Iniciar o Servidor

```
node server.js
```

Saída esperada: users-service listening on  
http://localhost:3010

## Testes com curl

Verificar saúde: curl  
http://localhost:3010/health

Criar usuário: curl -X POST  
http://localhost:3010/users -H 'Content-Type: application/json' -d  
'{"email":"aluno@example.com","name":"Aluno"}'